Stenographic File Integrity Checker

Team: Intern ID: 322,242,263
Environment: Kali Linux, Python 3.13, Virtual Environment (.venv)

1. Objective

The goal of this task was to build a tool that ensures the integrity of files by:

1. Generating a cryptographic hash of the target file.
2. Embedding the hash into a cover image using steganography (Least Significant Bit method).
3. Later extracting the hash to verify whether the original file has been modified.

This provides a lightweight method for detecting file tampering while keeping the hash hidden within an innocuous cover file.

2. Hashing Method

Cryptographic hashing is used to generate a fixed-length fingerprint of a file.

- **Algorithm:** SHA-256
- **Process:** The file is read in binary chunks, and a SHA-256 digest is computed.
- **Properties:**
    - Deterministic: the same file always produces the same hash.
    - Collision-resistant: extremely unlikely for two different files to produce the same hash.
    - Tamper-evident: any modification in the file changes the hash.

This hash serves as the integrity check value.

3. Steganography (LSB Method)

To hide the hash securely:

- **Method:** Least Significant Bit (LSB) embedding in PNG images.
- **Process:**
    - Each pixel's least significant bit (RGB channels) is used to store bits of the hash.
    - The cover image visually remains almost identical to the original.
    - The embedded hash can be extracted later for verification.
- **Tools Used:** Python stegano library with PIL (Python Imaging Library).

This ensures that the hash is hidden in plain sight, preventing easy discovery.

4. Workflow

1. **Embedding:**

   python embed.py report.pdf cover.png stego.png

Hash of report.pdf embedded into cover.png, saved as stego.png.

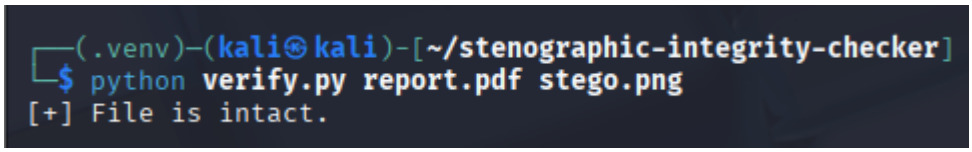
2. **Extraction:**

   python extract.py stego.png

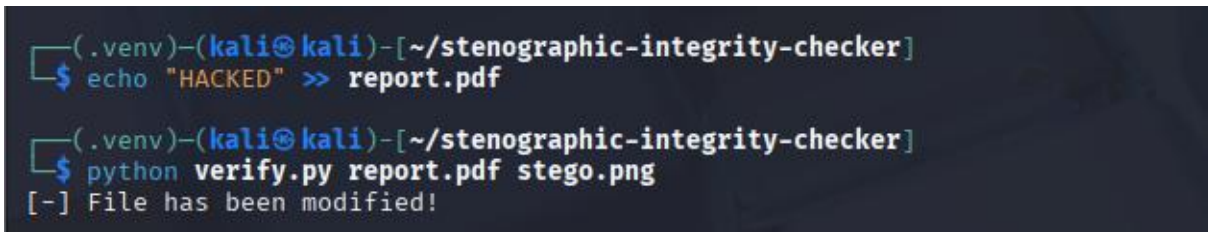
Retrieves the hidden hash.

3. **Verification:**

   python verify.py report.pdf stego.png



Compares current file hash with embedded hash → reports intact or modified.



5. Limitations

1. **File Types:** Currently only supports PNG images as cover files.
2. **Size Constraints:** Very large hashes or multiple files require proportionally large cover images.
3. **Library Dependency:** Relies on stegano for LSB embedding. Without it, custom implementation is needed.
4. **No Encryption:** Hidden hash is stored in plaintext inside the image; anyone with the image and knowledge of LSB could extract it.

6. Future Improvements

1. **Support Multiple File Types:** Allow embedding into JPG, BMP, or audio (WAV) files.
2. **Encryption:** Encrypt the hash before embedding for added security.
3. **Batch Processing:** Handle multiple files in a single run.
4. **GUI Tool:** Provide a user-friendly interface for embedding, extraction, and verification.
5. **Command-line Arguments:** Fully flexible filenames (already implemented in the upgraded scripts).
6. **Automated Reporting:** Generate logs or reports after each verification for audit purposes.